Logistic Regression, Gradient Descent & Assignment 1 Handout

Machine Learning 1 Practicals

Thomas Wedenig

March 25, 2026

Institute of Machine Learning and Neural Computation
Graz University of Technology, Austria

Table of contents

1. Linear Regression - Recap

2. Logistic Regression

3. Gradient Descent

4. Logistic Regression - Demo

5. Assignment 1 Handout

Linear Regression - Recap

- Inputs: features $x_1, x_2, \ldots, x_D$, collected in $\mathbf{x} = (x_1, x_2, \ldots, x_D)^T \in \mathbb{R}^D$
- Given $\mathbf{x}$, we want to predict a **real-valued target** $y \in \mathbb{R}$
- We collect a **training dataset** with many such input-output pairs

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(1)}, y^{(1)} \right), \ldots, \left(\mathbf{x}^{(N)}, y^{(N)} \right) \right\}$$

- **Linear Regression:** We assume an (approx.) **affine relationship** between $\mathbf{x}$ and y:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = b + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_D \cdot x_D$$

- With $\boldsymbol{\theta} = (b, w_1, w_2, \ldots, w_D)^T$ and $\mathbf{x} = (1, x_1, x_2, \ldots, x_D)^T$, we have

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}$$

- We can also use a **non-linear feature transformation** ϕ
- E.g., given a single feature $x \in \mathbb{R}$ we let $\phi(x) = (1 \;\; x \;\; x^2 \;\; x^3 \ldots \;\; x^6)^T$

$$f_{\boldsymbol{\theta}}(x) = \phi(x)^T \boldsymbol{\theta} = b + w_1 \cdot x + w_2 \cdot x^2 + \cdots + w_6 \cdot x^6$$

- The feature transform $\phi : \mathbb{R} \to \mathbb{R}^7$ lifts x into a higher dimensional space
- $f_{\boldsymbol{\theta}}(x)$ is still **linear in** $\boldsymbol{\theta} \in \mathbb{R}^7$

Logistic Regression

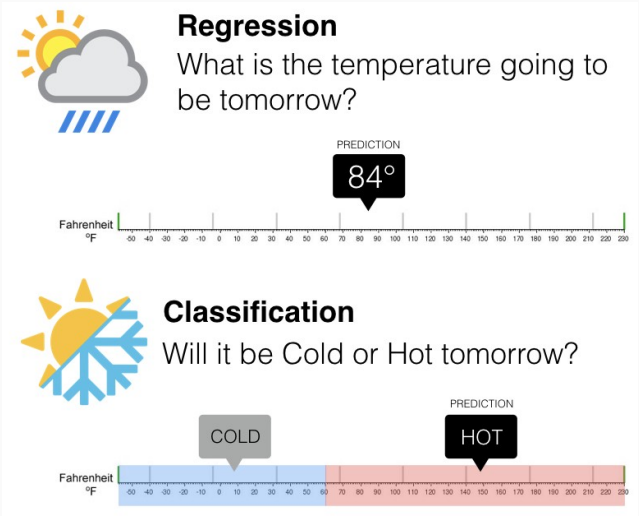
- We have a dataset $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$
- $\mathbf{x}^{(i)} \in \mathbb{R}^D$ and $y^{(i)} \in \mathcal{C}$
- $\mathcal{C}$ contains **discrete classes**, e.g. $\mathcal{C} = \{\texttt{cat}, \texttt{dog}, \texttt{rabbit}\}$
- **Binary Classification**: $\mathcal{C}$ contains **2 elements**

(Informal) Binary Classification Problem

Let $\mathcal{C} = \{-1, 1\}$. Given $\mathcal{D}$, find a function $g : \mathbb{R}^D \to \mathcal{C}$ that "predicts the labels well":

$$g(\mathbf{x}) \approx y$$

Regression vs. Classification



https://www.springboard.com/blog/data-science/regression-vs-classification/

Classification Example

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Male	Clicked on Ad
0	68.95	35	61833.90	256.09	0	0
1	80.23	31	68441.85	193.77	1	0
2	69.47	26	59785.94	236.50	0	0
3	74.15	29	54806.18	245.89	1	0
4	68.37	35	73889.99	225.58	0	0
5	59.99	23	59761.56	226.74	1	0
6	88.91	33	53852.85	208.36	0	0
7	66.00	48	24593.33	131.76	1	1

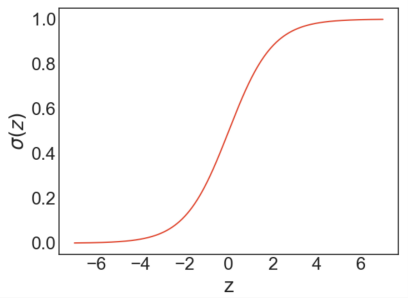
- **Logistic Regression**: A particular method for **binary classification problems**
- We want to **model the conditional probability** $p(y = 1 \mid \mathbf{x}) \in [0, 1]$
- Given $\mathbf{x}$, we use a **linear function**

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^T \phi(\mathbf{x})$$

  to compute a **logit** $\in (-\infty, \infty)$
- To convert the logit to a probability, we use the **(logistic) sigmoid function**:

$$p(y = 1 \mid \mathbf{x}) = \sigma(f_{\boldsymbol{\theta}}(\mathbf{x})), \qquad \sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$p(y = 1 \mid \mathbf{x}) = \sigma(\boldsymbol{\theta}^T \phi(\mathbf{x})), \qquad \sigma(z) = \frac{1}{1 + \exp(-z)}$$



- σ maps a real number $\in (-\infty, \infty)$ to the interval $[0, 1]$
- σ is **differentiable everywhere** and its gradients are **informative** 😍
- Finally, use the predicted probability to **make a decision**
  - If $p(y = 1 \mid \mathbf{x}) \geq 0.5$, predict $\hat{y} = +1$
  - If $p(y = 1 \mid \mathbf{x}) < 0.5$, predict $\hat{y} = -1$

- How to interpret the output of the **linear function** $f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^T \phi(\mathbf{x})$?
- Given $\mathbf{x}$, let $\pi_{\mathbf{x}} = p(y = 1 \mid \mathbf{x})$ be the **predicted probability of success**
- $f_{\boldsymbol{\theta}}(\mathbf{x})$ models the **log-odds**:

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \log\left(\frac{\pi_{\mathbf{x}}}{1 - \pi_{\mathbf{x}}}\right)$$

- From which follows that

$$\pi_{\mathbf{x}} = \sigma(f_{\boldsymbol{\theta}}(\mathbf{x}))$$

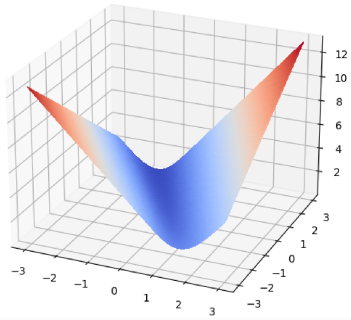
Assumption of Logistic Regression ❗

We assume a **linear** relationship between $\phi(\mathbf{x})$
and the **log-odds** of the event happening.

- (Binary) Cross-Entropy Loss Function:

$$\mathcal{L}_{CE}(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[y^{(i)} = 1] \log \pi_{\mathbf{x}^{(i)}} + \mathbb{1}[y^{(i)} = -1] \log (1 - \pi_{\mathbf{x}^{(i)}})$$

with $\pi_{\mathbf{x}^{(i)}} = \sigma(\boldsymbol{\theta}^T \phi(\mathbf{x}^{(i)}))$



- No closed form solution for $\boldsymbol{\theta}^* = \text{argmin}_{\boldsymbol{\theta}} \mathcal{L}_{CE}(\boldsymbol{\theta})$
- But $\mathcal{L}_{CE}(\boldsymbol{\theta})$ is **differentiable** and **convex** in $\boldsymbol{\theta}$ 😏
- Use some **optimizer** to solve for $\boldsymbol{\theta}^*$
  - e.g., Gradient Descent

- In ML, we **usually prefer "simple" models**
- We can encode this into the **loss function**
- **Assumption**: Simple models have parameter values in $\boldsymbol{\theta}$ that **are close to** 0
- Thus, we may **penalize** the model if the following grows large:

$$\sum_{i=1}^{D} \theta_i^2 = \|\boldsymbol{\theta}\|_2^2$$

- We now minimize

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{CE}(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2$$

where λ controls the **strength of regularization**

Gradient Descent

- Let $f(x_1, x_2, \ldots, x_D)$ be a **multi-variate scalar-valued** function ($f : \mathbb{R}^D \to \mathbb{R}$)

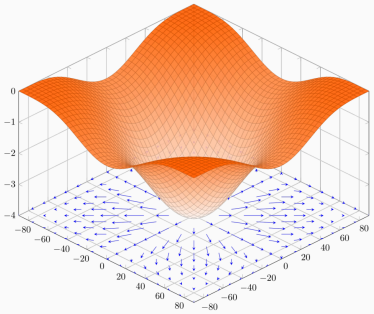
- The **gradient** of f w.r.t. $\mathbf{x} = (x_1, \ldots, x_D)^T$ at some point $\mathbf{x}$ is defined as

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_D} \end{pmatrix}$$

- Recall that

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_D) - f(\mathbf{x})}{h}$$

- $\nabla f(\mathbf{x})$ points in the direction of steepest ascent



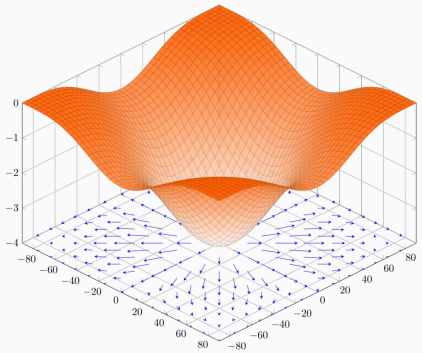
https://commons.wikimedia.org/wiki/File:3d-gradient-cos.svg

Gradient Descent (GD)

1. Select initial point $\mathbf{x}^{(0)}$ (e.g., randomly)
2. while $\|\nabla f(\mathbf{x}^{(t)})\|_2 > \varepsilon$ (for small $\varepsilon > 0$):

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \eta_t \nabla f(\mathbf{x}^{(t)})$$

with η_t denoting the **learning rate (step size)**



https://commons.wikimedia.org/wiki/File:3d-gradient-cos.svg

Logistic Regression - Demo

Assignment 1 Handout